



华中科技大学

虚拟存储器

许向阳

xuxy@hust.edu.cn

华中科技大学 计算机科学与技术学院



内 容

➤ 虚拟存储器的基本概念

➤ 虚拟存储器的实现

分页式虚拟存储器

分段式虚拟存储器

段页式虚拟存储器

逻辑地址、线性地址、物理地址

➤ 逻辑地址到线性地址的转换

➤ 线性地址到物理地址的转换

虚拟存储器的基本概念

- 程序员需要关心物理内存吗？
内存条有多大？
程序能全部放入内存吗？
程序放在内存的什么位置？

早期：程序员自己管理主存，通过分解程序并覆盖主存的方式执行程序

- 在实时性要求较高的嵌入式微控制器中，大多需要关注内存；
- 取指令和存储操作数所有的地址都是物理地址；
- 执行速度快，无需进行地址转换；
- 未采用虚拟存储机制。

虚拟存储器的基本概念

覆盖 (overlay)，是一种内存管理技术，用于在资源有限的环境中运行比物理内存更大的程序。

- 将程序**分解**为多个模块（主模块 + 多个覆盖模块），运行时按需加载特定模块到同一内存区域。
- 不同模块共享同一块内存空间，通过替换**覆盖**旧模块来节省内存（内存复用）。

```
int main() {    早期，程序员自己管控内存示例
    load_overlay("moduleA.ovl"); // 加载覆盖模块A
    run_function_A();
    load_overlay("moduleB.ovl"); // 替换为模块B
    run_function_B();
    return 0;
}
```

虚拟存储器的基本概念

- 1961年，英国曼切斯特研究人员提出**自动执行overlay**的方式。
- 动机：把程序员从大量繁琐的存储管理工作中解放出来，
使得程序员编程时不用管主存容量的大小。

基本思想

- 把**地址空间**和**主存容量**的概念区分开来；
- 程序员在逻辑地址空间里编写程序；
- 程序则在真正的内存中运行；
- 由一个**专门的机制**实现地址空间和实际主存之间的**映射**；
- CPU 中的 **MMU** (Memory Management Unit) 负责将**逻辑地址**（即虚拟地址）转换为 内存的**物理地址**。

虚拟存储器的基本概念

- 1961年，曼彻斯特大学的 Atlas 计算机首次实现了**虚拟内存** (Virtual Memory) 的基本概念
- 1965年，IBM System/360 Model 67 是首个商业计算机系统中明确引入类似 MMU 功能的机型，其核心是**动态地址转换** (Dynamic Address Translation, DAT)。这一硬件技术实现了**虚拟地址到物理地址的映射**，被认为是现代 MMU 的前身。
- 1969年，GE-645 计算机专为 Multics 操作系统（现代操作系统的先驱）设计，其硬件包含更成熟的**MMU** 功能，支持**分段** (Segmentation) 和**分页** (Paging) 机制。Multics 的研发进一步推动了 MMU 技术的标准化。
- 70 - 80年代，随着微处理器的发展，MMU 逐渐成为 CPU 的标准组件。

虚拟存储器的基本概念

- 虚拟存储技术的最初引入用来解决一对矛盾
 - 由于技术和成本等原因，主存容量受到限制
 - 系统程序和应用程序要求主存容量越来越大
- 虚拟存储技术的实质
 - 程序员在比实际主存空间大得多的逻辑地址空间中编写程序
 - 程序执行时，把当前需要的程序段和相应的数据块调入主存，其他暂不用的部分存放在磁盘上
 - 指令执行时，通过**硬件**将逻辑地址（也称虚拟地址或虚地址）转化为物理地址（也称主存地址或实地址）
 - 在发生程序或数据**访问失效(缺页)**时，**由操作系统**进行主存和磁盘之间的信息交换
- 虚拟存储器机制由硬件与操作系统共同协作实现，涉及到操作系统中的许多概念，如进程、进程的上下文切换、存储器分配、虚拟地址空间、缺页处理等。

虚拟地址

- 操作系统为每个进程分配独立的 **虚拟地址空间**
- 程序运行时感知的地址均为虚拟地址（如 0x00400000）
- 操作系统和硬件（MMU）负责映射到物理地址。

实验：编写一个程序，定义全局变量；用scanf 输入一个值给该全局变量，显示该变量的值。

同时运行该程序（两次），观察全局变量的地址是否相同？
地址相同，而对应单元中的内容不同，如何解释？

高级	设置校验和	否
调试	基址	
VC++ 目录	随机基址	是 (/DYNAMICBASE)
▷ C/C++	固定基址	是 (/FIXED)
▲ 链接器	数据执行保护(DEP)	是 (/NXCOMPAT)
常规	关闭程序集生成	否
输入	卸载延迟加载的 DLL	
清单文件	取消绑定延迟加载的 DLL	
调试	导入库	
系统	合并节	
优化	目标计算机	MachineX64 (/MACHINE:X64)
嵌入的 IDL	配置文件	否
Windows 元数据	CLR 线程特性	
高级	CLR 映像类型	默认映像类型
所有选项	密钥文件	

虚拟地址

- 固定基址 (Fixed Base Address)
- 随机基址 (Randomized Base Address)

是控制 程序在虚拟内存中加载位置 的两种关键选项。

特性	固定基址	随机基址 (ASLR)
地址确定性	✓ 固定, 可预测	✗ 随机, 不可预测
安全性	✗ 易受攻击	✓ 抵御内存攻击
加载性能	✓ 无重定位开销 (无冲突时)	⚠ 轻微重定位开销
兼容性	✗ 需避免基址冲突	✓ 动态适应地址空间
典型用途	内核驱动、遗留系统、高频动态库	现代应用程序、通用动态库

虚拟地址

固定基址 VS 随机基址

- 文件头信息差异
- 加载过程的不同
- 内存地址的确定性（指的是虚拟地址是否相同）
- 重定位表的存在
- 安全性的影响
- 性能差异
- 兼容性问题

旧系统或某些特定环境可能对固定基址有依赖，而现代系统更推荐随机基址。

- 调试和开发的便利性

固定基址可能在调试时更容易跟踪地址，而随机基址可能需要额外处理。

虚拟地址

两个程序同时运行，同时观察相同地址的内存

内存 1												
地址: 0x00411200										列: 自动		
0x00411200	13	00	00	e9	d8	19	00	00	e9	2a	25	... ??... ?*%
0x0041120B	00	00	e9	8e	18	00	00	e9	4a	25	00	.. ??... ?J%.
0x00411216	00	e9	a4	0d	00	00	e9	bf	1a	00	00	. ??... ??...
0x00411221	e9	1a	1a	00	00	e9	f5	1a	00	00	e9	?... ??... ?
0x0041122C	f5	24	00	00	e9	eb	25	00	00	e9	66	?\$. .. ??%. .. ?f
0x00411237	04	00	00	e9	11	0a	00	00	e9	8c	0b	? ??

内存 1												
地址: 0x00411200										列: 自动		
0x00411200	24	00	00	e9	6f	4b	00	00	e9	d4	4a	\$. .. ?oK.. ??J
0x0041120B	00	00	e9	1e	34	00	00	e9	40	4a	00	.. ?. 4.. ?@J.
0x00411216	00	e9	43	4b	00	00	e9	ef	15	00	00	. ?CK.. ??...
0x00411221	e9	1f	4a	00	00	e9	f5	4b	00	00	e9	?.. J.. ??K.. ?

程序调试时，看到的地址为虚拟地址

虚拟地址

➤ 实验

在 WSL (Windows Subsystem for Linux / Ubuntu) 下，
观察执行程序各个节的虚拟地址、各个变量的虚拟地址

```
gcc -g -Wl,-Ttext=0x8000 -Wl,-Tdata=0x10000  
data_test.c -o data_test
```

➤ `readelf -S -W data_test`

➤ `readelf -s data_test`

小结

虚拟存储器（Virtual Memory）是计算机系统中的一种核心内存管理技术，它通过软硬件结合的方式，让程序能够使用比物理内存（RAM）更大的逻辑地址空间。

核心思想是将物理内存和磁盘存储结合，为程序提供一种“无限内存”的假象，同时实现内存隔离、保护和高效共享。

虚拟存储器的实现

- **逻辑地址** (Logical Address) 是程序视角中的内存地址;
- **逻辑地址表达形式**与计算机架构和**内存管理模式**密切相关;
- **分页式** (纯分页模式, Paging)
 - 页号 : 页内偏移
- **分段式** (纯分段模式, Segmentation)
 - 段选择符 : 段内偏移
- **段页式** (Segmentation + Paging)
 - 分段: 逻辑地址转换为**线性地址** (段基址 + 偏移量)
 - 分页: 线性地址通过**页表**转换为物理地址
- **平坦模型** (Flat Memory Model) 现代 64 位系统
 - 逻辑地址 = 线性地址 = 虚拟地址

虚拟存储器的实现

1. 纯分段模式 (Segmentation)

- **典型架构**：早期 x86（实模式、保护模式）。
- **表达形式**：

段选择符 (**Segment Selector**) : 偏移量 (**Offset**)

- **段选择符**：16 位，指向段描述符表 (GDT/LDT) 中的条目，包含段基址和权限信息。
 - **偏移量**：16 位或 32 位（取决于模式），表示段内的具体位置。
- **转换过程**：

物理地址 = 段基址 (Base Address) + 偏移量

- **示例**：

x86 实模式下 **CS:IP**（代码段:指令指针）的地址 **0x1234:0x5678**，对应物理地址 **$0x1234 \times 16 + 0x5678 = 0x18AB8$** 。

虚拟存储器的实现

2. 纯分页模式 (Paging)

- **典型架构**：现代通用系统（如 Linux/Windows 默认模式）。

- **表达形式**：

虚拟页号 (Virtual Page Number, VPN) + 页内偏移 (Page Offset)

- **虚拟页号**：高位部分，用于索引页表 (Page Table)，找到对应的物理页帧号 (PFN)。
- **页内偏移**：低位部分，直接映射到物理页内的字节位置。

- **转换过程**：

物理地址 = PFN × 页大小 + 页内偏移

- **示例**：

32 位系统，页大小为 4KB (12 位偏移)：

逻辑地址 **0x12345678** → 分解为：

- VPN = **0x12345** (高 20 位)
- 偏移 = **0x678** (低 12 位)

通过页表查找到 PFN **0xABC**，物理地址为 **0xABC000 + 0x678 = 0xABC678**。

虚拟存储器的实现

3. 段页式结合 (Segmentation + Paging)

- **典型架构：**x86 保护模式（现代系统通常禁用分段，采用平坦模型）。
- **表达形式：**

段选择符：虚拟地址（分段后） → 线性地址（分页前）

- 分段阶段：逻辑地址转换为线性地址（如 段基址 + 偏移量）。
- 分页阶段：线性地址通过页表转换为物理地址。

- **示例：**

x86 保护模式下，逻辑地址 **DS:0x5678** 经过分段得到线性地址 **0x12345678**，再分页转换为物理地址。

虚拟存储器的实现

4. 平坦模型 (Flat Memory Model)

- **典型架构**：现代 64 位系统（如 x86-64、ARMv8）。

- **表达形式**：

单一连续的逻辑地址空间，无需段选择符，直接分页。

- 逻辑地址 = 线性地址 = 虚拟地址（分段被弱化或禁用）。

- **示例**：

x86-64 中逻辑地址直接为 64 位虚拟地址（高 16 位通常未使用），如 `0x7FFE12345678`。

- ❑ 在x86-64架构中，段寄存器（CS、DS等）仍存在
- ❑ 段被设置为覆盖整个地址空间；
- ❑ 代码段和数据段的基地址均为0，（基地址=0）；
- ❑ 段限长=最大地址；
- ❑ 逻辑地址直接等于线性地址，从而消除分段的影响

分页 (Paging)

◦ 基本思想：

如：页大小=页框大小=4KB

- 内存被分成固定长且比较小的存储块 (页框、实页、物理页)
- 每个进程也被划分成固定长的程序块 (页、虚页、逻辑页)
- 程序块可装入主存页框中
- 无需用连续页框存放一个进程
- 操作系统为每个进程生成一个页表
- 通过页表(page table)实现逻辑地址向物理地址转换

◦ 逻辑地址 (Logical Address) :

- 程序中指令所用地址(进程所在地址空间),
也称为虚拟地址 (Virtual Address, 简称VA)

◦ 物理地址 (Physical Address, 简称PA) :

- 存放指令或数据的实际主存地址, 也称为实地址、主存地址。

分页 (Paging)

问题：是否需要将一个进程的全部都装入内存？为什么？

根据程序访问局部性可知：可把当前活跃的页面调入主存，其余留在磁盘上！

采用“按需调页 Demand Paging”方式分配主存！这就是虚拟存储管理概念

VPN: 虚页号
VPO: 页内地址

PPN: 页框号
PPO: 页内地址

VA: $VPN + VPO$

PA: $PPN + PPO$

逻辑地址



物理地址



进程 A 页表

0	13
1	14
2	15
3	18

页表描述了虚拟页和物理页框之间的映射关系！

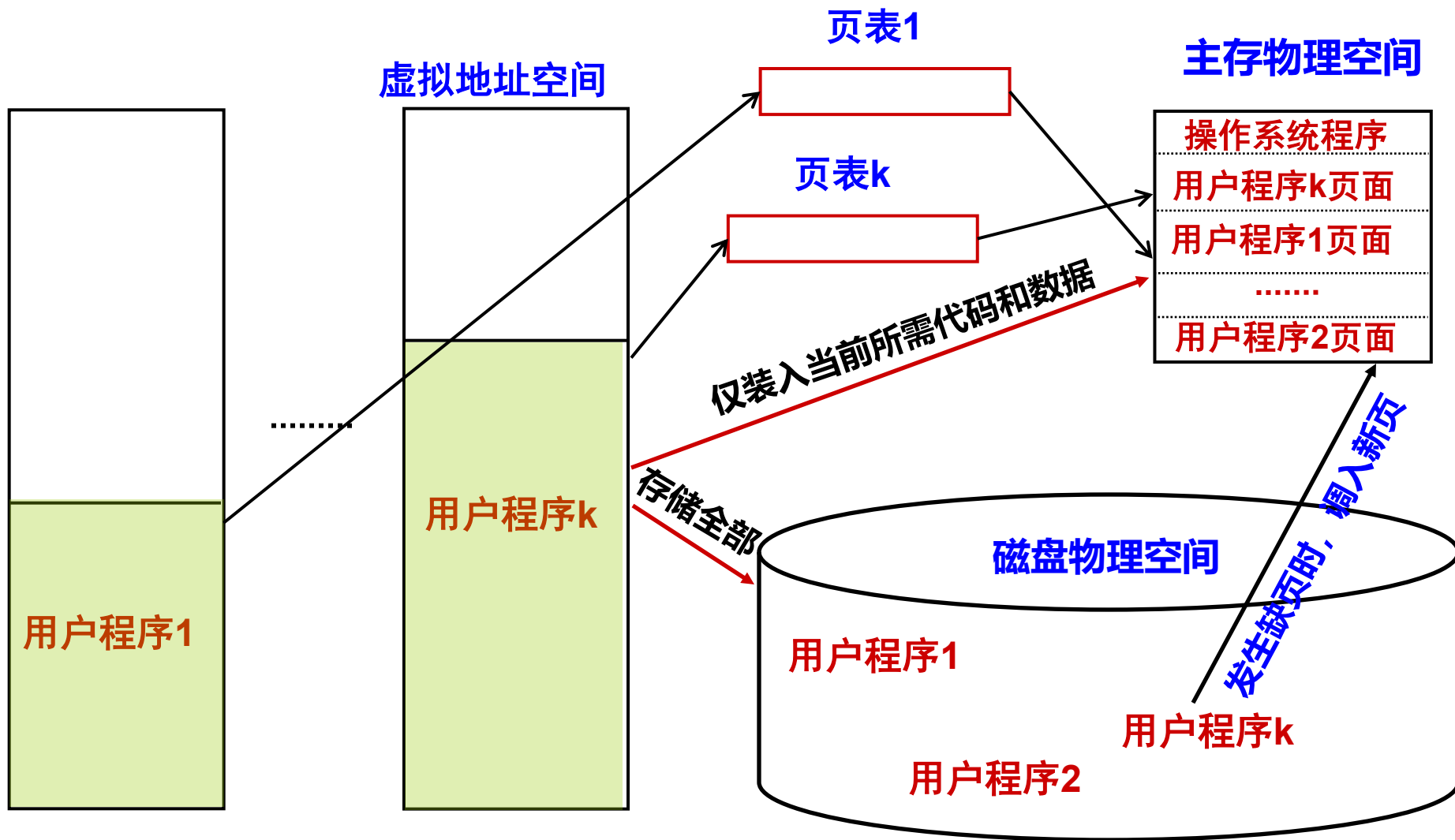


主存

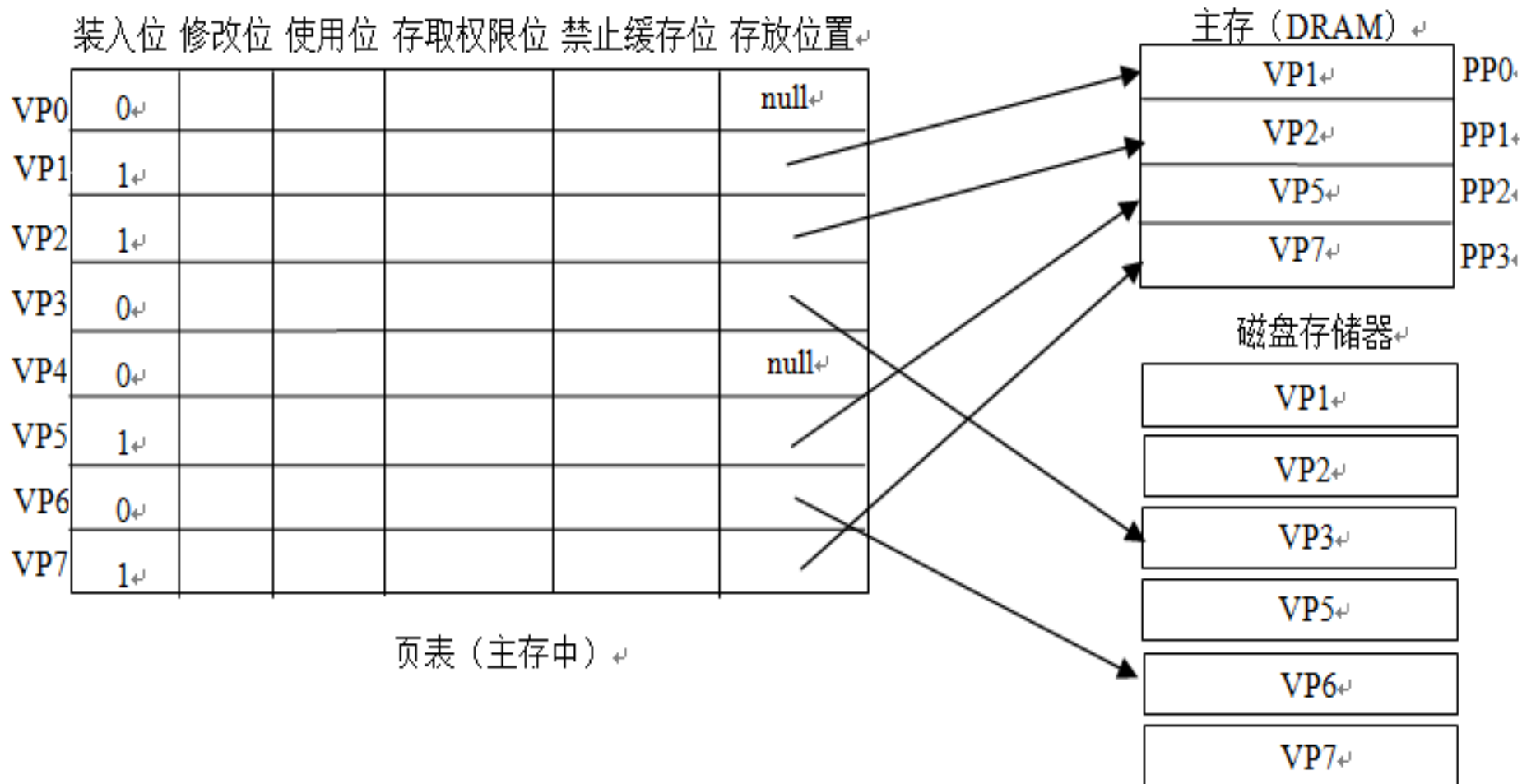
**页大小有什么特点？和页内地址位数有什么关系？
地址转换由硬件还是软件实现？为什么？**

分页 (Paging)

通过页表建立虚拟空间和物理空间之间的映射!

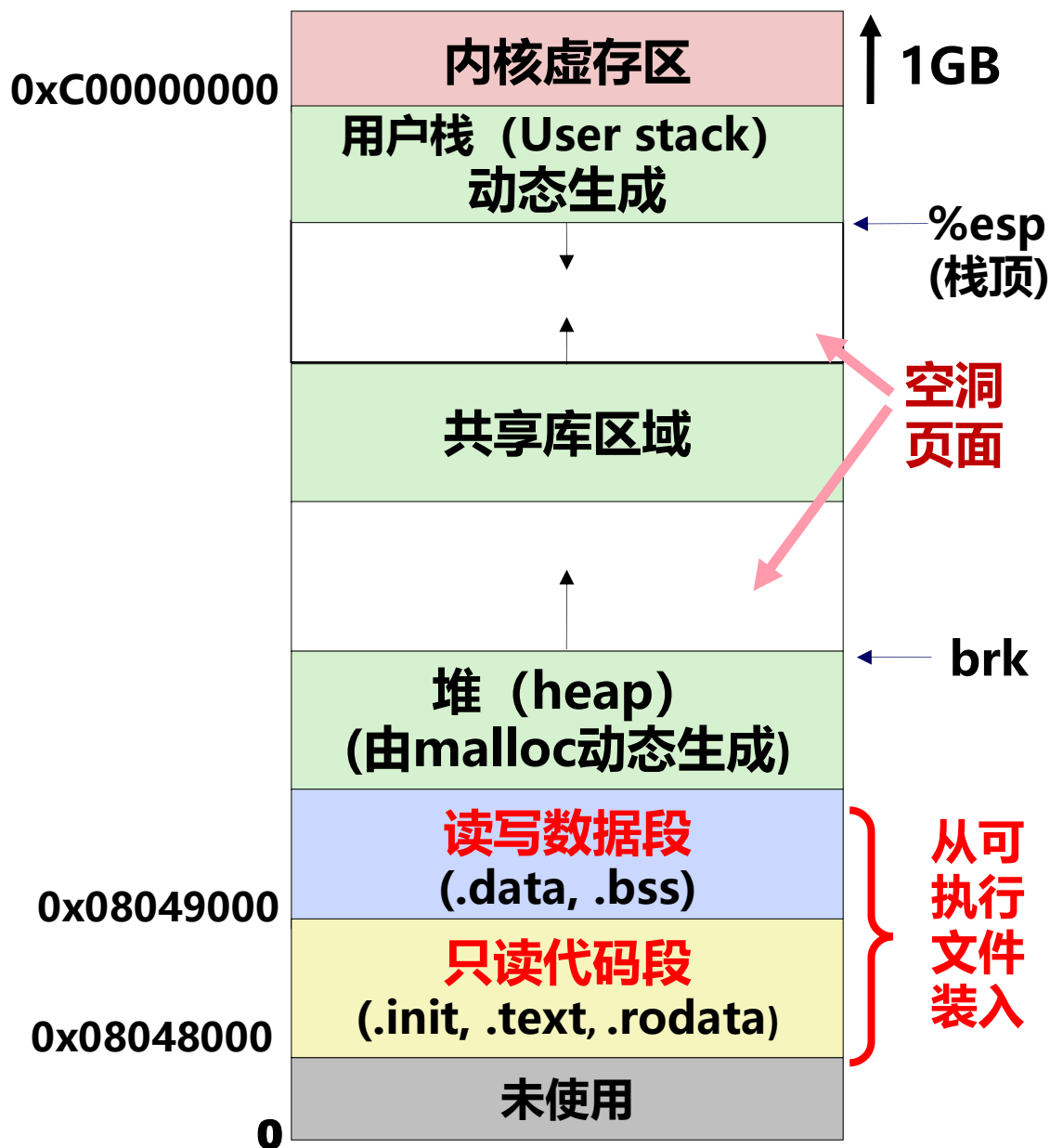


主存中的页表示例

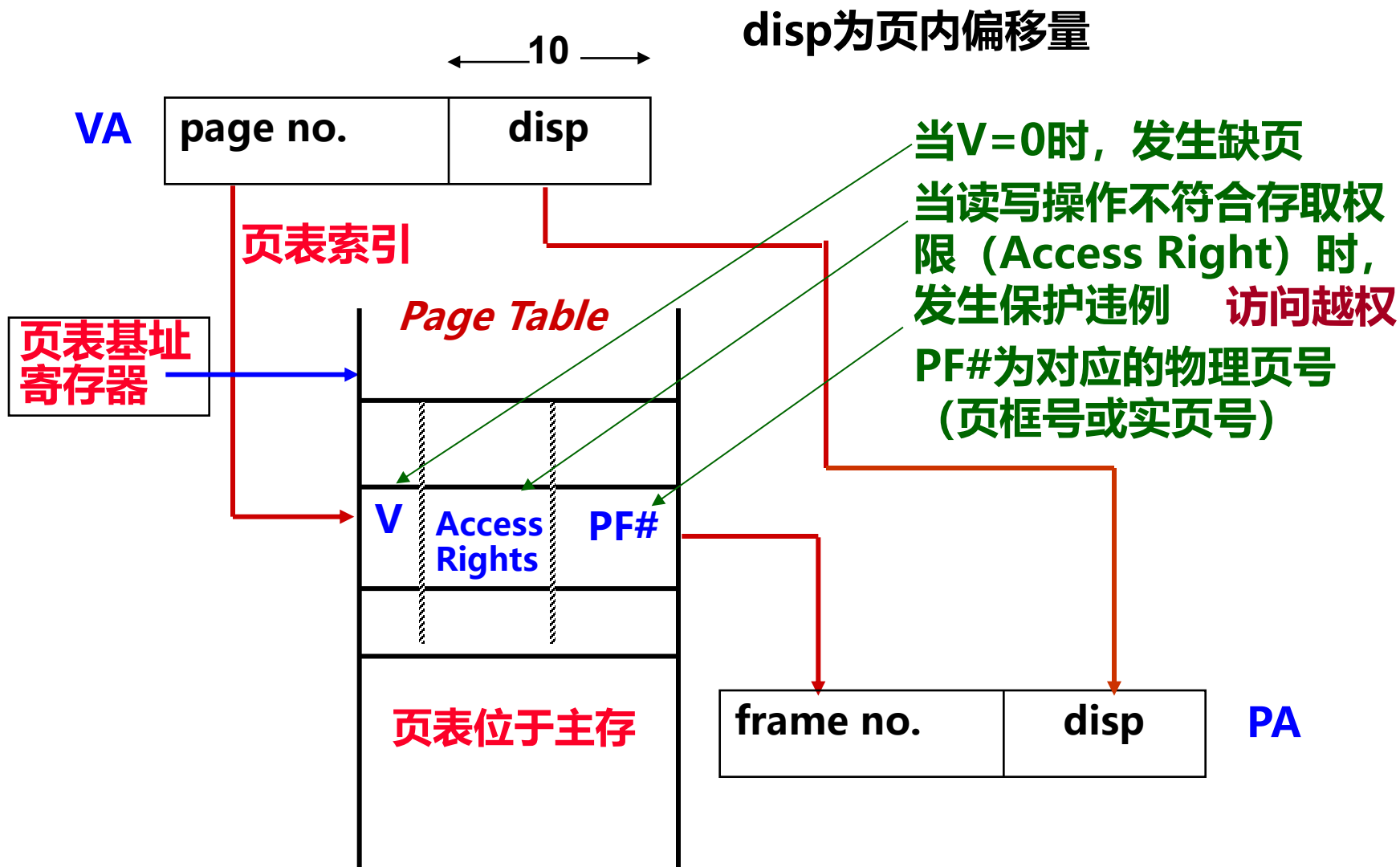


- ◆ **未分配页**: 进程的虚拟地址空间中“空洞”对应的页 (如VP0、VP4)
- ◆ **已分配的缓存页**: 有内容对应的已装入主存的页 (如VP1、VP2、VP5等)
- ◆ **已分配的未缓存页**: 有内容对应但未装入主存的页 (如VP3、VP6)

空洞页面



逻辑地址转换为物理地址的过程



问题1：什么情况下，不能成功进行地址转换呢？

问题2：如何知道页表中的存取权限应该设置成什么（R/W、R、X）？

信息访问中可能出现的异常情况

可能有两种异常情况：

1) 缺页 (page fault)

产生条件：当Valid (有效位 / 装入位) 为 0 时

相应处理：从磁盘读页面到主存，若主存没有空间，则从主存选择一页替换到磁盘上，替换算法类似于Cache，采用回写法，淘汰时，根据“dirty”位确定是否要写磁盘

当前指令执行被阻塞，当前进程被挂起，处理结束回到原指令继续执行

2) 保护违例 (protection_violation_fault) 或访问违例

产生条件：当Access Rights (存取权限)与所指定的具体操作不相符时

相应处理：在屏幕上显示“内存保护错”或“访问违例”信息

当前指令的执行被阻塞，当前进程被终止

Access Rights (存取权限)可能的取值有哪些？

R = Read-only, R/W = read/write, X = execute only

分段式虚拟存储器

- ° 分页方式：不同信息跨页 或 浪费空间
- ° 分段系统的实现
 - 程序有多个代码段和数据段构成，按照程序的逻辑结构划分，而形成多个相对独立的部分。
(例如，代码段、只读数据段、可读写数据段等，按实际大小划分)
 - 段带有段名或基地址，便于程序编写、编译器优化和操作系统调度管理
 - 分段系统将主存空间按实际程序中的段来划分，每个段在主存中的位置记录在段表中，并附以“段长”项
 - 段表由段表项组成，段表本身也是主存中一个可再定位段

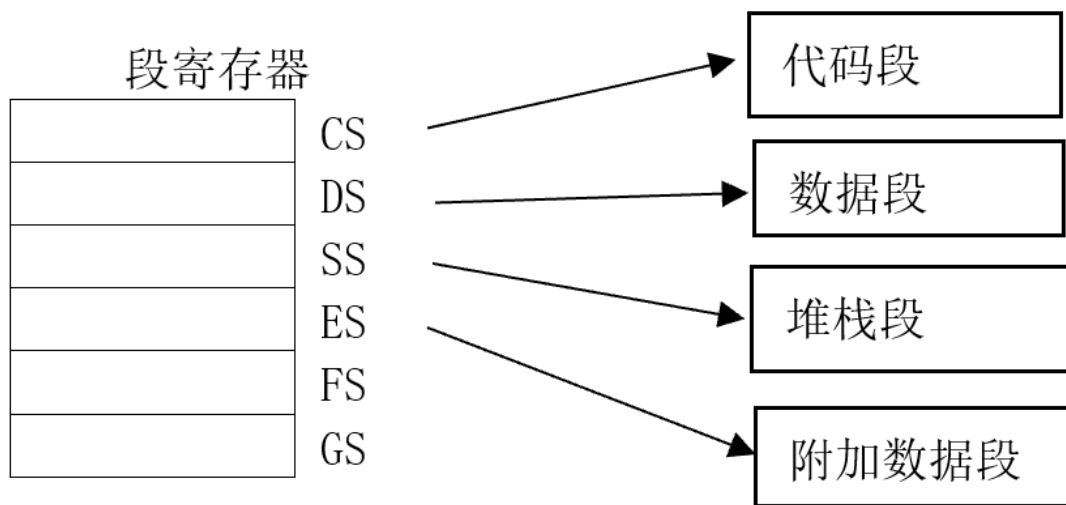
内核虚存区
用户栈 (User stack) 动态生成
共享库区域
堆 (heap) (由malloc动态生成)
读写数据段 (.data, .bss)
只读代码段 (.init, .text, .rodata)
未使用

分段式虚拟存储器的地址映像



分段式虚拟存储器

- 在**分段内存模型**中，每个分段通常加载不同的分段选择器，以便每个段寄存器指向线性地址空间内的不同段。



分段式虚拟存储器

Q: 要保护一个段, 应该提供哪些信息?
这些信总又存放在何处?

描述符 (description)

段的起始位置 (段基地址)

段的大小 (段界限)

段的特权级

段的属性 (是代码段, 数据段, 还是堆栈段?)

(数据段是否可写? 代码段是否可读?)

段的位置 (在内存还是在磁盘?)

段的类型 (在系统段还是用户段?)

段的使用 (段被访问过, 还是没有?)

分段式虚拟存储器

段描述符

段的有关信息的描述



段基地址： B31 ~ B0, 共32位；

段界限: L19 ~ L0, 共20位

分段式虚拟存储器

描述符表

描述符的集合 —— 描述符表

局部描述符表:

一个LDT, 是一个系统段, 最大可为64KB, 最多可存放8192个描述符。(64KByte / 8Byte per Descriptor)

对每一个程序, 都建立一个局部描述符表 (LDT)。

LDT_A

描述符A0



描述程序A的代码段

描述符A1



描述程序A的数据段

描述符A2



描述程序A的堆栈段



描述程序A的.....

LDT_B

描述符B0



描述程序B的代码段

描述符B1



描述程序B的数据段



描述程序B的.....

Local Description Table

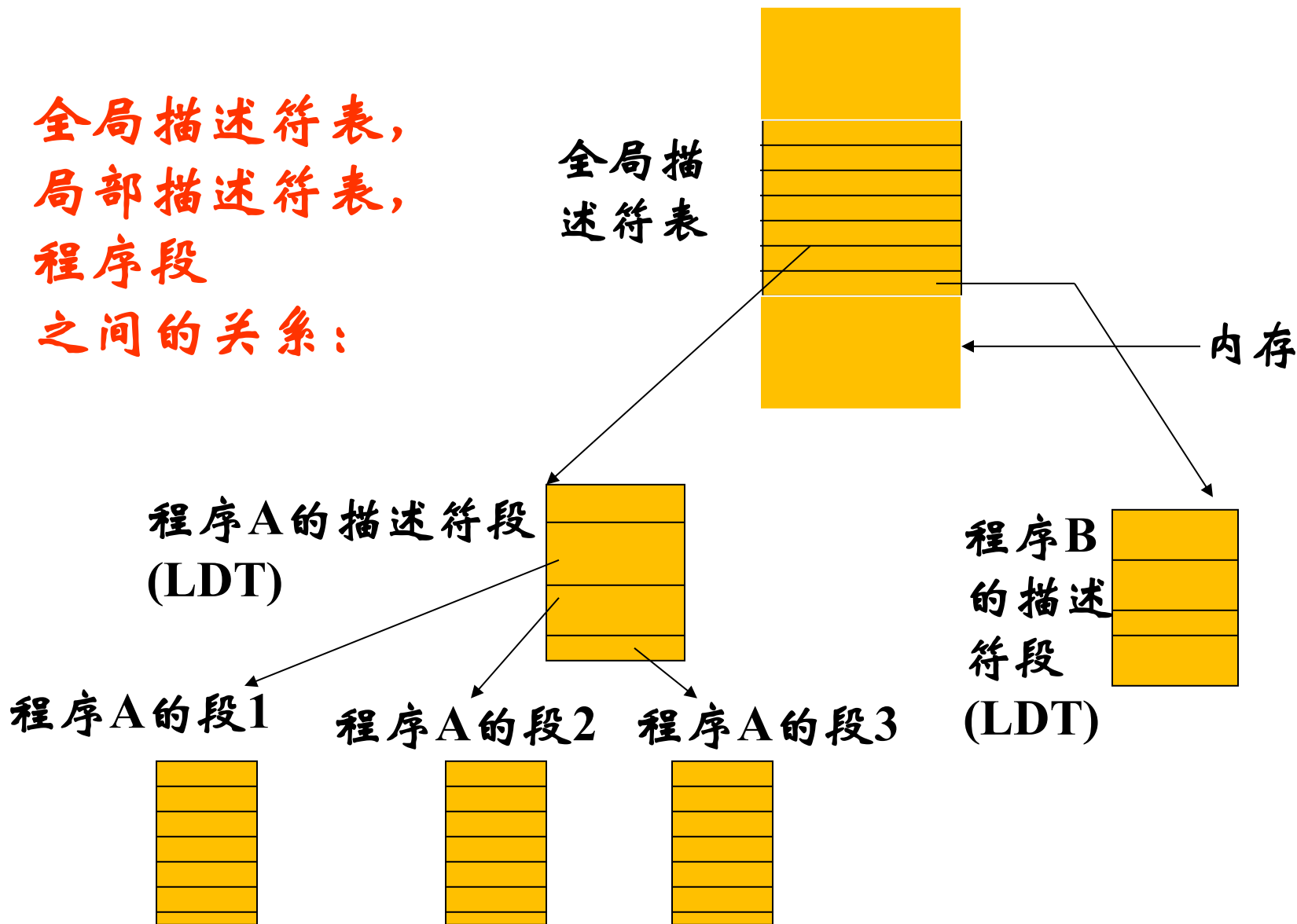
分段式虚拟存储器

全局描述符表： 只有一个。GDT最大可为64KB，
存放8192个描述符。包括：

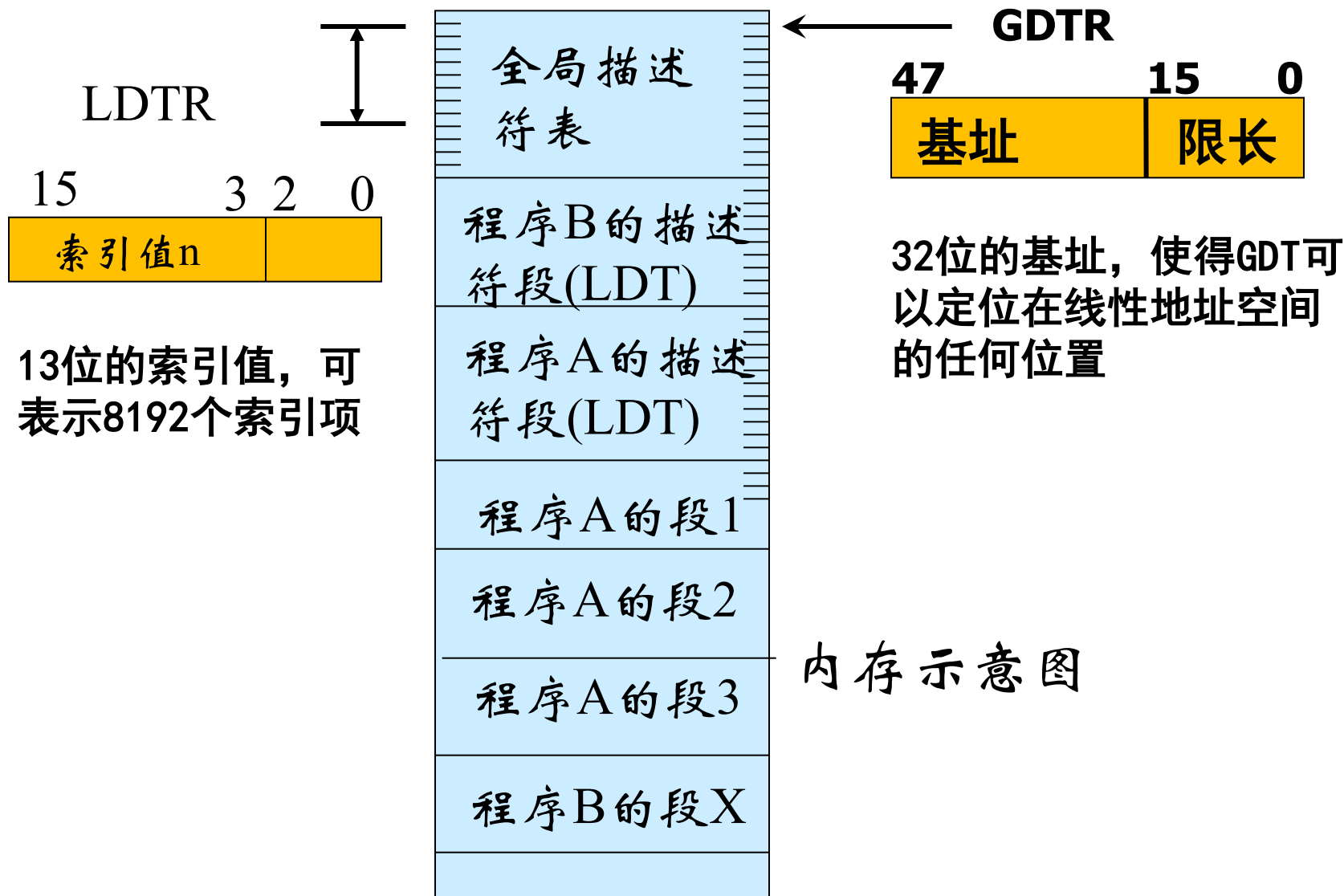
- 操作系统所使用的段的描述符；
- 各个LDT段的描述

GDT	OS代码段描述符	→	描述OS的代码段
	OS数据段描述符	→	描述OS的数据段
	OS堆栈段描述符	→	描述OS的堆栈段
	LDT_A段的描述符	→	描述LDT_A段
	LDT_B段的描述符		

分段式虚拟存储器

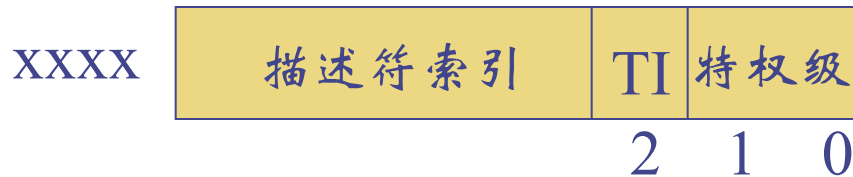


分段式虚拟存储器



xxxx: cs、ds、ss、es、fs、gs

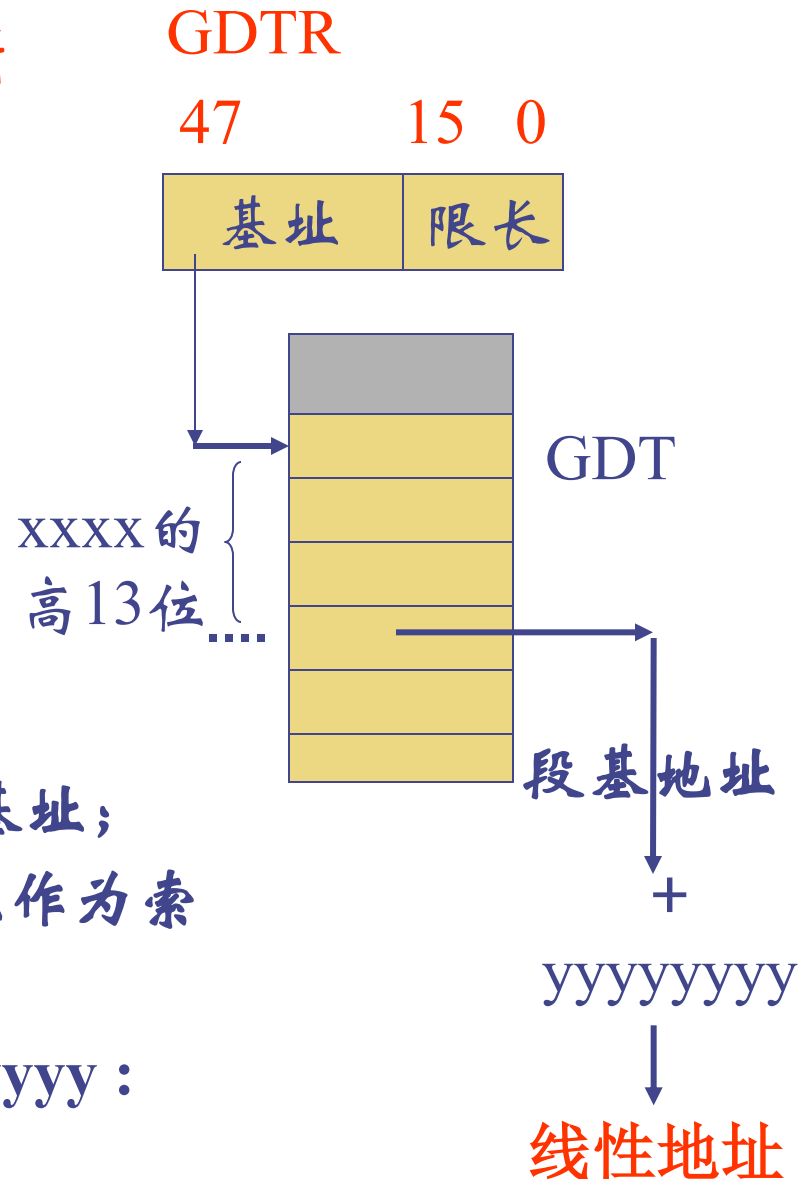
xxxx : yyyyyyyy 到线性地址的映射



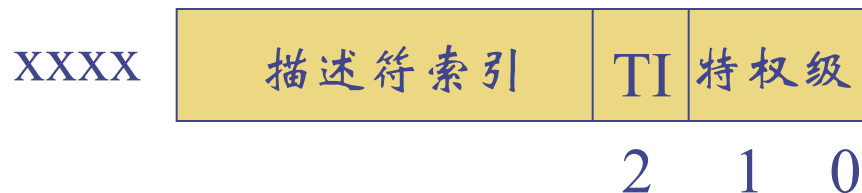
XXXX不是段的开始地址，
而是指出找相应段描述符
的方式。称为**段选择符**。

(1) TI 位为 0

- 从GDTR寄存器中获取GDT的基址；
- 在GDT表中，以XXXX的高13位作为索引，取出一个描述符A；
- 描述符A中的段基地址 + yyyyyyyy：
为要访问单元的线性地址。

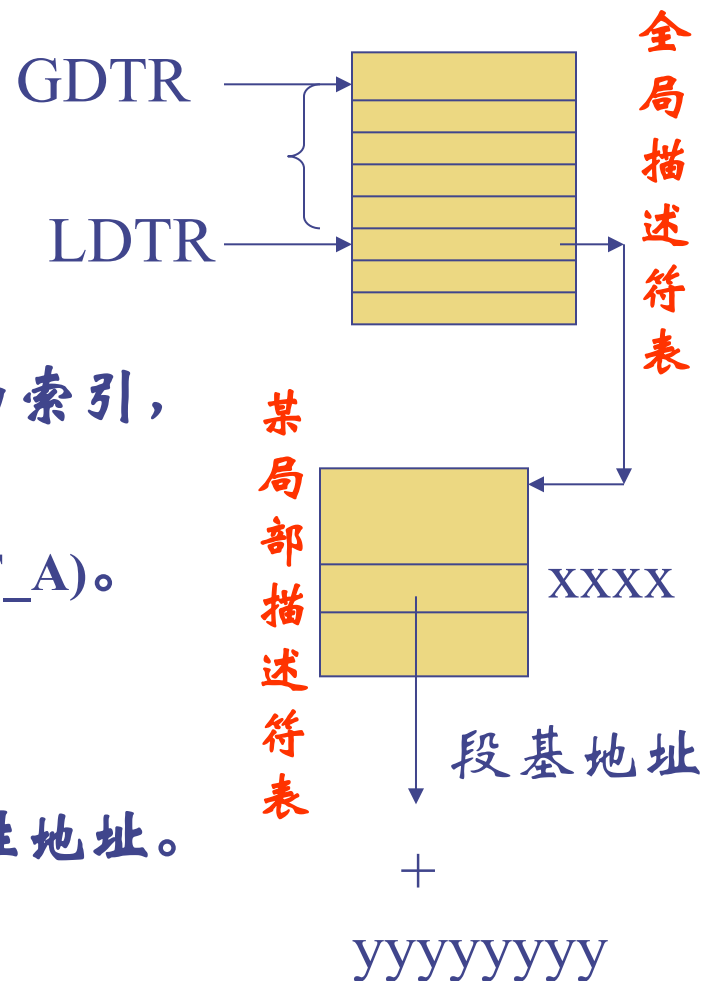


xxxx : yyyyyyyyyy 到线性地址的映射

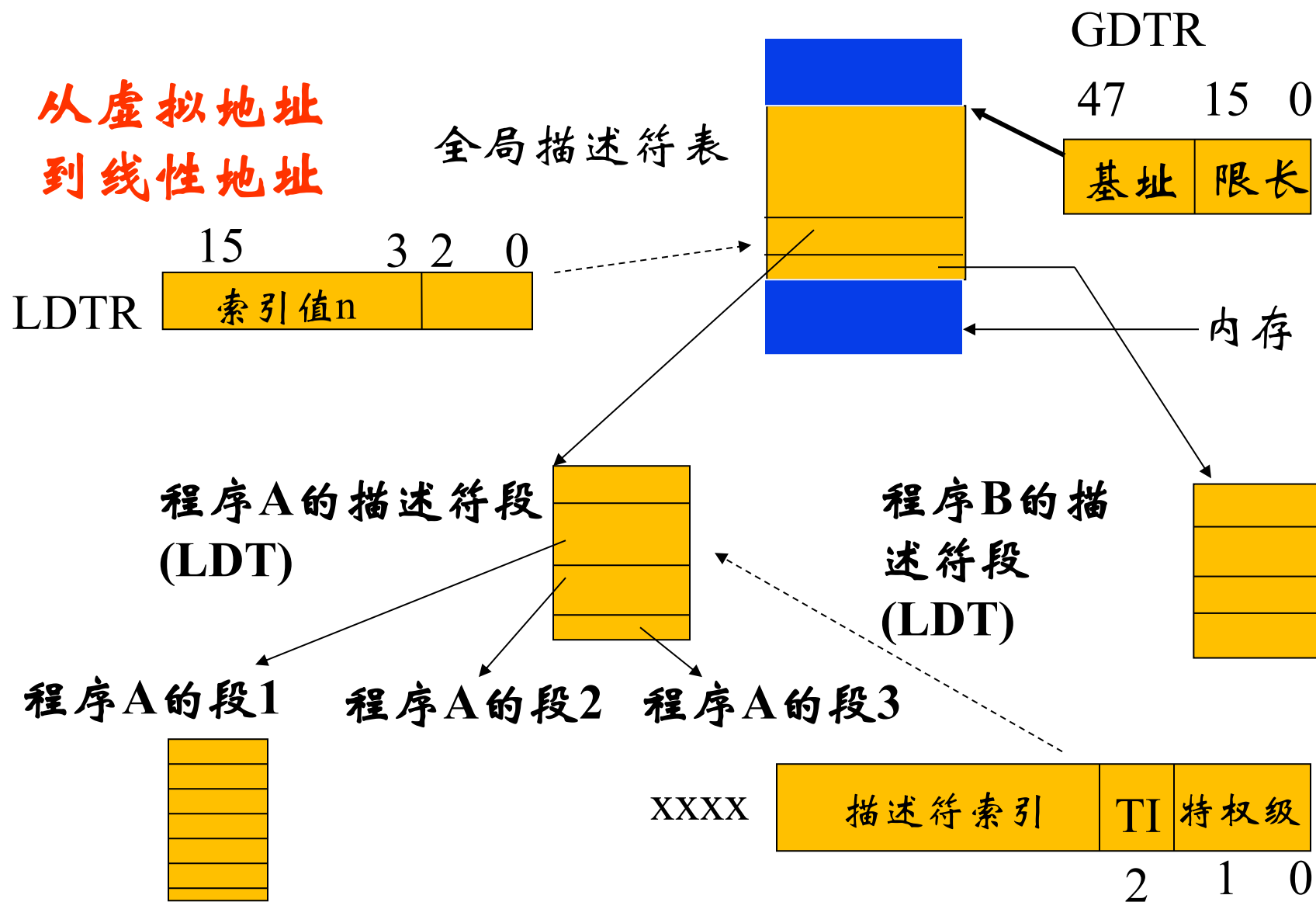


(2) 若TI 位为 1

- 从GDTR寄存器中获取GDT的基址；
- 在GDT表中，以LDTR的高13位作为索引，取出一个描述符A；
- 描述符A描述的段为一个LDT段(LDT_A)。
- 用 XXXX的高13位，作为索引，在LDT_A段中找到描述符P_A。
- P_A描述段的基址 + yyyyyyyyyy 为线性地址。



分段式虚拟存储器



分段式虚拟存储器

Intel® 64 and IA-32 Architectures
Software Developer's Manual

Q: GDTR 是如何设置的?

LGDT/LIDT—Load Global/Interrupt Descriptor Table Register

Opcode	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
OF 01 /2	LGDT <i>m16&32</i>	M	N.E.	Valid	Load <i>m</i> into GDTR.
OF 01 /3	LIDT <i>m16&32</i>	M	N.E.	Valid	Load <i>m</i> into IDTR.
OF 01 /2	LGDT <i>m16&64</i>	M	Valid	N.E.	Load <i>m</i> into GDTR.
OF 01 /3	LIDT <i>m16&64</i>	M	Valid	N.E.	Load <i>m</i> into IDTR.

Description

Loads the values in the source operand into the global descriptor table register (GDTR) or the interrupt descriptor table register (IDTR). The source operand specifies a 6-byte memory location that contains the base address (a linear address) and the limit (size of table in bytes) of the global descriptor table (GDT) or the interrupt descriptor table (IDT). If operand-size attribute is 32 bits, a 16-bit limit (lower 2 bytes of the 6-byte data operand) and a 32-bit base address (upper 4 bytes of the data operand) are loaded into the register. If the operand-size attribute is 16 bits, a 16-bit limit (lower 2 bytes) and a 24-bit base address (third, fourth, and fifth byte) are loaded. Here, the high-order byte of the operand is not used and the high-order byte of the base address in the GDTR or IDTR is filled with zeros.

The LGDT and LIDT instructions are used only in operating-system software; they are not used in application programs. They are the only instructions that directly load a linear address (that is, not a segment-relative address) and a limit in protected mode. They are commonly executed in real-address mode to allow processor initialization prior to switching to protected mode.

In 64-bit mode, the instruction's operand size is fixed at 8+2 bytes (an 8-byte base and a 2-byte limit). See the summary chart at the beginning of this section for encoding data and limits.

分段式虚拟存储器

Q: LDTR 是如何设置的?

LLDT—Load Local Descriptor Table Register

Opcode	Instruction	Op/En	64-Bit Mode	Compat/Leg Mode	Description
0F 00 /2	LLDT <i>r/m16</i>	M	Valid	Valid	Load segment selector <i>r/m16</i> into LDTR.

Description

Loads the source operand into the segment selector field of the local descriptor table register (LDTR). The source operand (a general-purpose register or a memory location) contains a segment selector that points to a local descriptor table (LDT). After the segment selector is loaded in the LDTR, the processor uses the segment selector to locate the segment descriptor for the LDT in the global descriptor table (GDT). It then loads the segment limit and base address for the LDT from the segment descriptor into the LDTR. The segment registers DS, ES, SS, FS, GS, and CS are not affected by this instruction, nor is the LDTR field in the task state segment (TSS) for the current task.

If bits 2-15 of the source operand are 0, LDTR is marked invalid and the LLDT instruction completes silently. However, all subsequent references to descriptors in the LDT (except by the LAR, VERR, VERW or LSL instructions) cause a general protection exception (#GP).

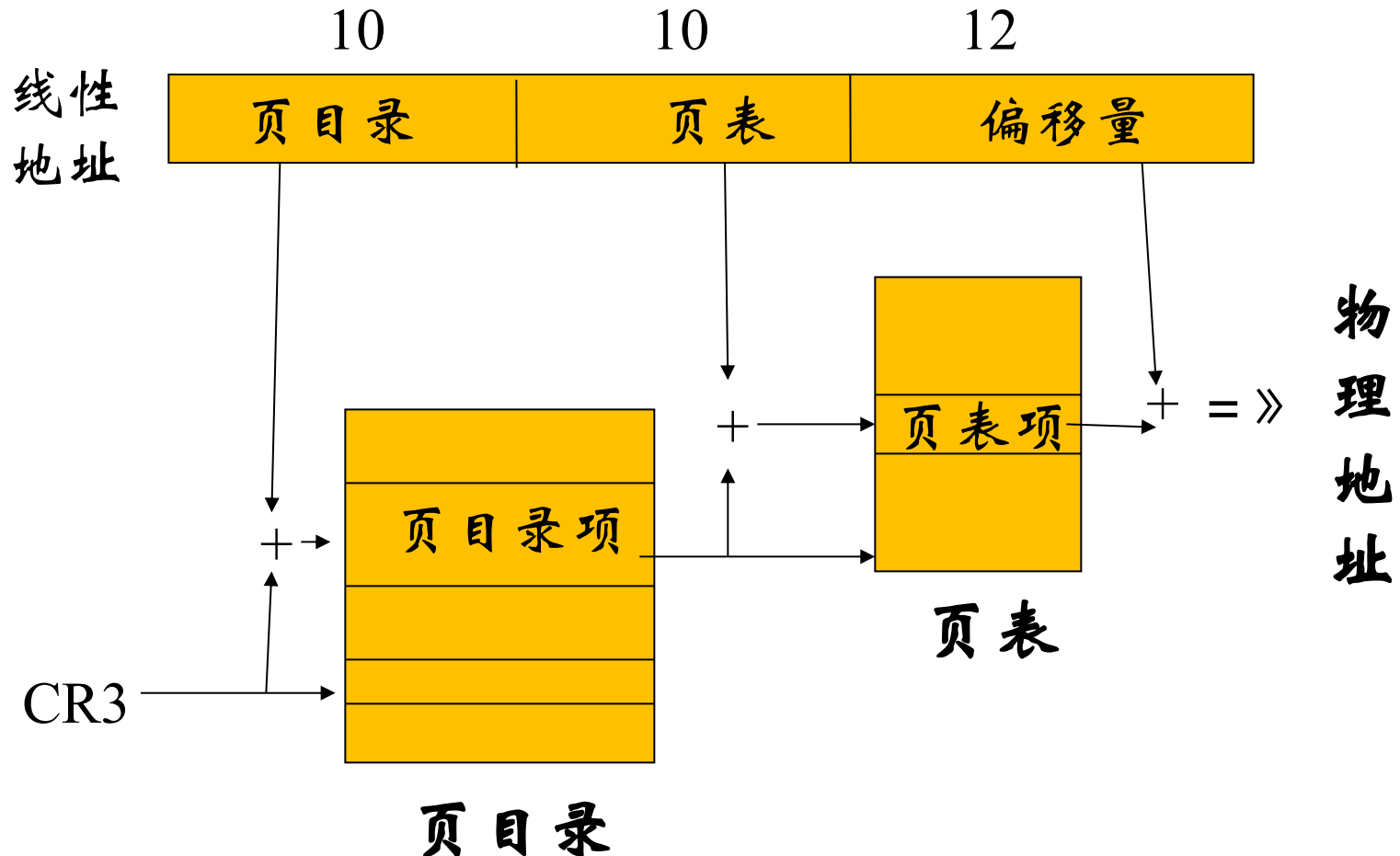
The operand-size attribute has no effect on this instruction.

The LLDT instruction is provided for use in operating-system software; it should not be used in application programs. This instruction can only be executed in protected mode or 64-bit mode.

In 64-bit mode, the operand size is fixed at 16 bits.

从线性地址到物理地址

从线性地址到物理地址



IA-32的存储管理

- 按字节编址（通用计算机大都是）
- 在保护模式下，IA-32采用**段页式**虚拟存储管理方式
- 存储地址采用逻辑地址、线性地址和物理地址来进行描述，其中，**逻辑地址和线性地址是虚拟地址的两种不同表示形式，描述的都是4GB虚拟地址空间中的一个存储地址**
 - ✓ 逻辑地址由48位组成，包含16位段选择符和32位段内偏移量（即**有效地址**）
 - ✓ 线性地址32位（其位数由虚拟地址空间大小决定）
 - ✓ 物理地址32位（其位数由存储器总线中的地址线条数决定）
- 分段过程实现将逻辑地址转换为线性地址
- **分页过程实现将线性地址转换为物理地址**

总 结

- 虚拟地址空间
- 虚拟地址空间的管理
- 描述符
- 描述符表
 - 局部描述符表、全局描述符表、中断描述符表
- 段选择符
- 线性地址的形成
- 从虚拟地址到物理地址的转换